



Developing DTL Transformations

Version 2026.1
2026-04-20

Developing DTL Transformations

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Introduction to DTL Tools	1
1.1 Background	1
1.2 Available Tools in the Management Portal	1
1.3 Other Tools	1
1.4 Using Data Transformations	2
1.5 See Also	2
2 Introduction to the DTL Editor	3
2.1 Displaying the DTL Editor	3
2.2 A Look at the User Interface	3
2.3 See Also	5
3 Creating Data Transformations	7
3.1 Building DTLs During a Migration	7
3.2 Creating a Transformation	7
3.3 Opening an Existing Transformation	8
3.4 Specifying Transformation Details	8
3.4.1 When to Use an Existing Object As the Target	10
3.5 Using the DTL Explainer	10
3.6 Undoing and Redoing Changes	11
3.7 Saving a Transformation	11
3.8 Compiling a Transformation	11
3.9 Deleting a Transformation	11
3.10 See Also	11
4 Adding and Editing Actions	13
4.1 Adding an Action	13
4.2 Editing an Action	13
4.3 Rearranging Actions	13
4.4 See Also	14
5 DTL Syntax Rules	15
5.1 References to Message Properties	15
5.2 Literal Values	15
5.2.1 XML Reserved Characters	16
5.2.2 Separator Characters in Virtual Documents	16
5.2.3 When XML Reserved Characters Are Also Separators	16
5.2.4 Numeric Character Codes	17
5.3 Valid Expressions	17
5.4 See Also	18
6 Actions That Set or Clear Values	19
6.1 Introduction	19
6.1.1 Objects and Object References	19
6.2 Adding a SET Action	19
6.3 Shortcuts for Adding a SET Action	20
6.3.1 Copying the Source Message	20
6.3.2 Copying a Value from a Source Property to a Target Property	20
6.4 Using the Function Wizard	21
6.5 SET and Collections	22

6.6 Adding an INSERT Action	22
6.7 Adding an APPEND Action	22
6.8 Adding a REMOVE Action	23
6.9 REMOVE and Collections	23
6.10 Clearing a Collection Property	24
6.11 See Also	24
7 Other Actions	25
7.1 Adding an IF Action	25
7.2 Adding a FOR EACH Action	26
7.2.1 Shortcuts for the FOR EACH Action	27
7.2.2 Unloading Target Collections	27
7.2.3 Avoiding <STORE> Errors with Large Messages	28
7.3 Adding a SUBTRANSFORM Action	28
7.4 Adding a TRACE Action	29
7.5 Adding a CODE Action	29
7.5.1 Guidelines for Using Custom Code in DTL	30
7.6 Adding an SQL Action	30
7.6.1 Guidelines for Using SQL in DTL	30
7.7 Adding a SWITCH Action	31
7.8 Adding a CASE Action	31
7.9 Adding a Default Action	32
7.10 Adding a Break Action	32
7.11 Adding a COMMENT Action	32
7.12 See Also	32
8 Listing and Managing Data Transformations	33
8.1 Introduction	33
8.2 Options on This Page	33
8.3 Related Options	34
8.4 See Also	34
9 Testing Data Transformations	35
9.1 Using the Transformation Testing Page	35
9.2 Testing a Transformation Programmatically	36
9.3 See Also	36
DTL Reference	37
DTL <annotation>	38
DTL <assign>	39
DTL <break>	42
DTL <case>	43
DTL <code>	44
DTL <comment>	46
DTL <default>	47
DTL <>false>	48
DTL <foreach>	49
DTL <group>	51
DTL <if>	52
DTL <sql>	53
DTL <subtransform>	54
DTL <switch>	56
DTL <trace>	57

DTL <transform>	58
DTL <true>	60
Appendix A: Configuring the DTL Explainer	61
A.1 On-prem Systems	61
A.2 Cloud-based Systems	61
A.3 See Also	61

1

Introduction to DTL Tools

DTL transformations are a form of [business logic](#) you can use within [interoperability productions](#). This topic introduces the tools that InterSystems IRIS® data platform provides to enable you to develop and test DTL transformations.

1.1 Background

A *data transformation* creates a new message that is a transformation of another message. It is common for a production to use data transformations, to adjust outgoing messages to the requirements of the target systems.

You can create and edit a *DTL transformation* visually in the DTL Editor, available in either the Management Portal or [your IDE](#). The DTL Editor is meant for use by nontechnical users. The term *DTL* represents Data Transformation Language, which is the XML-based language that InterSystems IRIS uses internally to represent the definition of a transformation that you create in this editor.

You can invoke a data transformation from a business process, another data transformation, or a business rule. Note that there is overlap among the options available in business processes, data transformations, and business rules. For a comparison, see [Comparison of Business Logic Tools](#). You can also try using these tools yourself by [Creating a Data Transformation](#).

1.2 Available Tools in the Management Portal

The Management Portal provides the following tools for working with data transformations:

- The [DTL Editor](#), which enables you to create, edit, and compile DTL transformations.
- The [Data Transformation List page](#), which enables you to test, import, export, and delete either kind of data transformation. It also enables you to open a DTL transformation in the DTL Editor.

1.3 Other Tools

You can also invoke a data transformation programmatically, which can be useful for testing purposes. See [Testing Data Transformations](#).

Also, because data transformations are classes, you can edit them and work with them in the same way that you do any other class.

1.4 Using Data Transformations

You can invoke a data transformation from the following parts of a production:

- From another DTL data transformation. See [Adding a Subtransform Action](#).
- From a BPL business process. See [<transform>](#).
- From a business rule. See [Passing Data to a Data Transformation](#).
- From a custom business process or a custom DTL transformation. To do so, execute it programmatically as described in [Testing Data Transformations](#).

Note: This section applies to both DTL transformations and custom transformations.

1.5 See Also

- [Comparison of Business Logic Tools](#)
- [Introduction to the DTL Editor](#)
- [Creating Data Transformations](#)
- [Adding and Editing Actions](#)
- [DTL Syntax Rules](#)
- [Listing and Managing Data Transformations](#)
- [Testing Data Transformations](#)

2

Introduction to the DTL Editor

The DTL Editor enables you to create, edit, and compile DTL transformations.

Important: After a period of inactivity, the InterSystems Management Portal may log you out and discard any unsaved changes. Inactivity is the time between calls to the InterSystems IRIS server. Not all actions constitute a call to the server. For example, clicking **Save** constitutes a call to the server, but typing in a text field does not. Consequently, if you are editing a data transformation, but have not clicked **Save** for longer than **Session Timeout** threshold, your session will expire and your unsaved changes will be discarded. After a logout, the login page appears or the current page is refreshed. For more information, see [Automatic Logout Behavior](#) in the Management Portal.

For information on performing these tasks with the legacy UI, see [Creating Data Transformations \(Legacy UI\)](#).

2.1 Displaying the DTL Editor

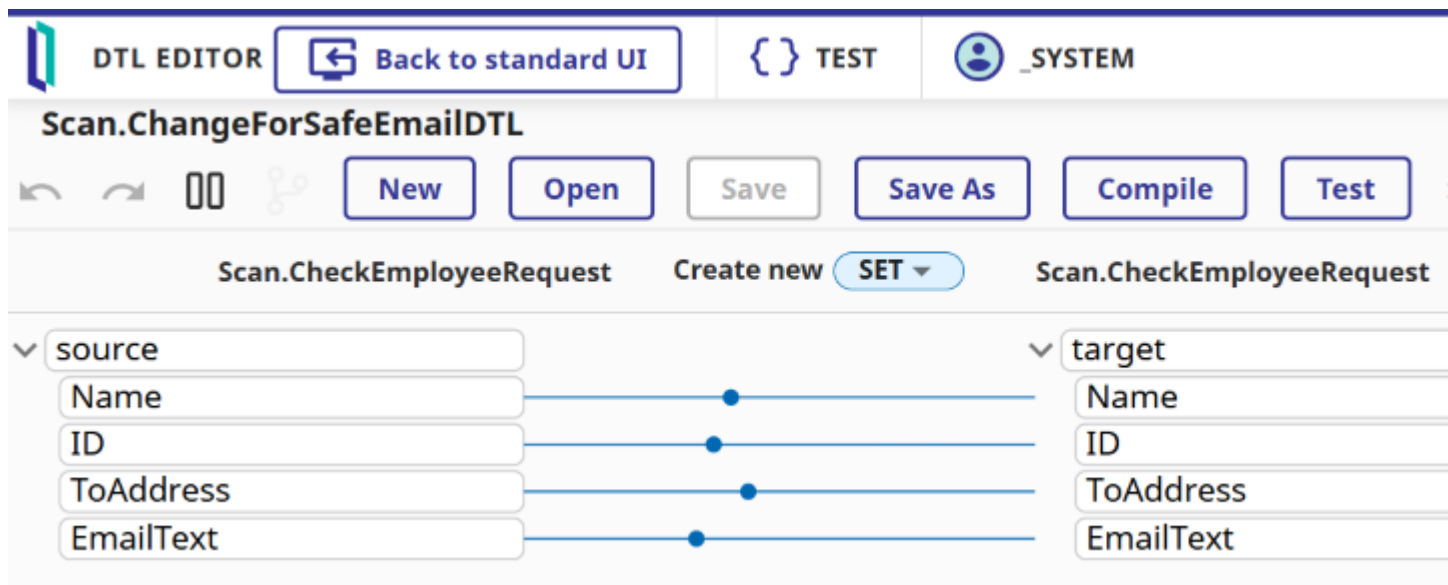
To access this page in the Management Portal:

1. Select **Interoperability > Build > Data Transformations**.
2. Click **Try the new UI**.

You can also access the DTL Editor from the [Production Configuration](#) page.

2.2 A Look at the User Interface

When first displayed, the DTL Editor contains two areas. The upper area displays the source and target messages, along with options that enable you to work with the data transformation as a whole:



Notice the name of the DTL being edited (Scan.ChangeForSafeEmailDTL in this example) and the names of the source and target message classes (Scan.CheckEmployeeRequest in both cases, in this example). This area of the page is meant to provide a quick visual overview of the DTL. In most cases, a DTL copies or modifies parts of the source message class to the target message class, and the connection lines indicate this visually.

The menu bar provides options you can use to do the following:

- Undo or redo your most recent change, via the buttons on the top left.
- Display the two parts of the page side by side, via the Side by Side  button.
- Create a new DTL transform or open an existing one (via the **New** and **Open** buttons, respectively).
- Save the transform or save it to a new name (via the **Save** and **Save As** buttons, respectively).
- Compile the transform (via the **Compile** button).
- Display the test page (via the **Test** button).
- Display other details for the transformation, via the gear icon on the right.

In this area, via drag and drop, you can add new actions—specifically the kinds of actions that modify values. To do so, select an option from **Create new** and then hover the cursor over a field in the source message and drag to a field in the target message; you can create **SET**, **APPEND**, **INSERT**, **CLEAR**, and **REMOVE** actions.

The bottom area of the page shows the actual DTL transformation, which is an ordered list of actions. In the following example, the DTL contains four actions.

Actions (0 selected)









 Disable

☐	1	SET	target.EmailText = "Contains gizmo "_source.EmailText
☐	2	SET	target.ID = source.ID
☐	3	SET	target.Name = source.Name
☐	4	SET	target.ToAddress = \$piece(source.ToAddress,"@",2)

The menu bar of this area allows you to make more detailed changes to the DTL. Here you can do the following:

- Cut, copy, and paste actions from one spot to another.
- Delete actions.
- Move actions to earlier or later parts of the DTL.
- Disable actions.
- Create new actions, via the **New** dropdown. In contrast to the button in the upper area, this dropdown lets you create any kind of action. When you select an option from this dropdown, a dialog box prompts for details.

When you select an action in this list, the display changes so that you can make edits.

2.3 See Also

- [Introduction to DTL Tools](#)
- [Creating Data Transformations](#)
- [DTL Syntax Rules](#)
- [Listing and Managing Data Transformations](#)
- [Testing Data Transformations](#)
- [Introduction to the Production Configuration Page](#)

3

Creating Data Transformations

This topic describes generally how to create and edit [data transformations](#) for [interoperability productions](#).

You can also visit Online Learning to try [Creating a Data Transformation](#).

Note: For information on the legacy DTL Editor, see [Introduction to the Legacy Editor](#).

3.1 Building DTLs During a Migration

If you intend to migrate to InterSystems IRIS for Health from other vendors, and you have a body of existing source and target messages, you can streamline the process of creating the necessary DTLs:

- You can automatically generate a starting DTL that performs the simpler transformations.
- You can compare the target messages output from the generated DTL to the original target messages, quickly identifying message segments that require attention.

To build a post-migration DTL using an existing body of source messages and target messages, follow the instructions at [DTL Generator](#).

3.2 Creating a Transformation

To create a transformation:

1. Display the [DTL Editor](#).
2. Click **New**.

InterSystems IRIS then displays a dialog box where you can specify the basic information for the transformation.

3. Specify some or all of the following information:

- **Package** (required)—Enter a package name.

Do not use a reserved package name; see [Reserved Package Names](#).

- **Name** (required)—Enter a name for your data transformation class.
- **Description**—Enter an description for the data transformation; this becomes the class description.

- **Source Class**—Specifies the type of messages that this transformation will receive as input.

If you are using a common input type, click one of the following options:

- **All Messages**—This transformation can be used with any input message type.
- **HL7**—The input messages are instances of `EnsLib.HL7.Message`.
- **X12**—The input messages are instances of `EnsLib.EDI.X12.Document`.
- **ASTM**—The input messages are instances of `EnsLib.EDI.ASTM.Document`.
- **EDIFACT**—The input messages are instances of `EnsLib.EDI.EDIFACT.Document`.
- **XML**—The input messages are instances of `EnsLib.EDI.XML.Document`.

Otherwise click the search icon and then select the class.

- **Source Document Type** (applicable only if the messages are virtual documents)—Enter or choose the document type of the source messages. You can choose any type defined in the applicable schemas loaded into this namespace.
- **Target Type** and **Target Class**—Specifies the type of messages that this transformation will generate as output. See the choices for **Source Class**.
- **Target Document Type** (applicable only if the messages are virtual documents)—Enter or choose the document type of the target messages. You can choose any type defined in the applicable schemas loaded into this namespace.

Apart from **Package** and **Name**, you can edit all these details later.

4. Specify details on the **Transform** tab. See [Specifying Transformation Details](#).
5. Then [add actions](#) as needed.

3.3 Opening an Existing Transformation

To open a transformation:

1. Display the [DTL Editor](#).
2. Click **Open**.

If you are currently viewing a transformation and you have made changes but have not yet saved them, InterSystems IRIS prompts you to confirm that you want to proceed (which will discard those changes).

3. Click the package that contains the transformation.

Then click the subpackage as needed.

4. Click the transformation class.

3.4 Specifying Transformation Details

To see details that define the transformation as a whole:

1. Display the [DTL Editor](#).

2. Click the Settings icon  .

These details are as follows:

- **DTL Name** (read-only)—Complete package and class name of the data transformation class.
- **Description**—Description of the data transformation. You can edit this manually or you can generate a description from the DTL diagram; see [Using the DTL Explainer](#).
- **Source Class**—Specifies the type of messages that this transformation will receive as input. For details, see [Creating a Transformation](#).
- **Source Document Type** (applicable only if the messages are virtual documents)—Specifies the document type of the source messages.
- **Target Class**—Specifies the type of messages that this transformation will generate as output. For details, see [Creating a Transformation](#).
- **Target Document Type** (applicable only if the messages are virtual documents)—Specifies the document type of the target messages.
- **Mode**—Specifies how the transformation should create the target message. Choose one of the following:
 - **Create new**—Create a new object of the target class (and type, if applicable), before executing the elements within the data transformation. This is the default.
 - **Copy**—Create a copy of the source object to use as the target object, before executing the elements within the transform.
 - **Existing**—Use an existing object, provided by the caller of the data transformation, as the target object. See the [following subsection](#).
- **Report Errors**—Specifies whether InterSystems IRIS should log any errors that it encounters when executing this transform. If you select this option, InterSystems IRIS logs the errors as *Warnings* in the Event Log. InterSystems IRIS also returns a composite status code containing all errors as its return value. This option is selected by default.
- **Treat empty repeating fields as null**—Specifies whether InterSystems IRIS skips the following actions for repeating fields when the fields are empty:
 - **foreach** actions—If you select this option, InterSystems IRIS does not execute **foreach** actions on repeating fields that are empty.
 - **assign** actions—If you select this option, InterSystems IRIS does not execute **assign** actions on repeating fields if you use shortcut notation to indicate that both the `source` and `target` fields are repeating fields, and the `source` field is empty. For example, if the `source.{PV1:AdmittingDoctor() }` field is empty and you select this option, then InterSystems IRIS does not execute the following action:


```
<assign value='source.{PV1:AdmittingDoctor() }'  
property='target.{PV1:AdmittingDoctor() }' action='set'.
```

 However, InterSystems IRIS *does* execute the following similar action since the `target` field is not a repeating field:


```
<assign value='source.{PV1:AdmittingDoctor() }'  
property='target.{PV1:AdmittingDoctor(1) }' action='set' />
```

 This option is cleared by default.
- **Allow empty segments in target**—Specifies whether to ignore errors caused by attempts to get field values out of absent source segments of virtual documents or properties of objects. If you select this option, InterSystems IRIS suppresses these errors and does not call subtransforms where the named source is absent. This option is selected by default.

You can precisely control the behavior by including tests and conditional logic branches to confirm that any required elements are present.

- **Language**—Specifies the language you will use in any expressions in this DTL. This can be **python** or **objectscript**.
- **Python From/Import Statements**; Specifies an optional list of Python `from` / `import` statements, one per line. Use this so that Python code within this DTL can refer to these modules.

3.4.1 When to Use an Existing Object As the Target

For **Mode**, the **Existing** option enables you to specify the target as an existing object, which results in a performance improvement. This option applies when you invoke a series of transformations programmatically (or perform other sequential processing). You would use this option in cases like the following scenario:

- You have three transformations that you want to perform in sequence:
 1. MyApp.ADTTransform—Uses the **Create New** option for **Mode**.
 2. MyApp.MRNTransform—Uses the **Existing** option for **Mode**.
 3. MyApp.LabXTransform—Uses the **Existing** option for **Mode**.
- You invoke the transforms as follows:

```
do MyApp.ADTTransform.Transform(message, .target)
do MyApp.MRNTransform(target, .newtarget)
do MyApp.LabXTransform(newtarget, .outmessage)
```

3.5 Using the DTL Explainer

If your system is [configured](#) to include the DTL Explainer, there is a **Generate New** button next to the **Description** text box. This button enables you to generate a description of the DTL. The DTL Explainer is an AI tool that examines the DTL logic and creates a description summarizing the contents of that logic. You can use this description as part or all of your description, and you can edit it if needed. You can also discard it.

Note: This is an AI Tool. Errors may occur. See [Disclaimer](#) for more information.

To use the DTL Explainer:

1. Click **Generate New**.

The system then displays a read-only generated description, including a disclaimer.

If the DTL is large, the processing may take some time.
2. If there is currently no text in the **Description** field, the system displays the generated description. Now you can either:
 - To close the generated description without saving it, click the X in the upper right.
 - To insert the generated description, scroll to the bottom and click **Insert**.

If there is current text in **Description** field, you have more options:

- To close the generated description without saving it, click the X in the upper right.
- To replace the description with the generated description, scroll to the bottom and click **Replace**.
- To insert the generated description before the existing description, scroll to the bottom and click **Insert** > **Prepend**.

- To insert the generated description after the existing description, scroll to the bottom and click **Insert > Append**.

3. Make and save any additional edits.

If you save the generated description, it is fully editable and it no longer contains the disclaimer.

3.6 Undoing and Redoing Changes

To undo the previous change, click the Undo button .

To redo the previous change, click the Redo button .

3.7 Saving a Transformation

To save a transformation, do one of the following:

- Click **Save**.
- Click **Save As**. Then specify a new package, class name, and description and click **OK**.
- Click **Compile**. This option saves the transformation and then compiles it.

3.8 Compiling a Transformation

To compile a transformation, click **Compile**. This option saves the transformation and then compiles it.

3.9 Deleting a Transformation

To delete a transformation, you use a different page in the Management Portal:

1. In the Management Portal, click **Interoperability > List > Data Transformations**.
2. Click the row that displays its name.
3. Click the **Delete** button.
4. Click **OK** to confirm this action.

3.10 See Also

- [Introduction to the DTL Editor](#)
- [Adding and Editing Actions](#)

- [Listing and Managing Data Transformations](#)
- [Testing Data Transformations](#)

4

Adding and Editing Actions

This page describes generally how to add, edit, and rearrange actions within a DTL transformation.

For information on performing these tasks with the legacy UI, see [Adding and Editing Actions \(Legacy UI\)](#).

4.1 Adding an Action

To add an action, do the following in the **Actions** area:

1. Decide where to place the new action, and click the action just before where you want the new action to be included.
2. Click the **New** menu and then select an action.

The DTL Editor adds the new action below the action you had selected.

3. Edit the details for the new action.

Other techniques are possible for **assign** actions, as discussed in [Actions That Set or Clear Values](#).

4.2 Editing an Action

To edit an action, first select it. To do so, do one of the following:

- If the DTL diagram displays the action, click the icon on the corresponding connector line.
- Click the item in the **Actions** area.

Now edit the values in the **Actions** area. Optionally, you can disable the action by selecting it and then clicking the **Disabled** check box. If you disable a **FOR EACH** or **IF** action, all actions within the block are also disabled.

4.3 Rearranging Actions

InterSystems IRIS executes the actions in the order they are listed in the **Actions** area.

To rearrange actions, use the **Actions** area:

1. Click the check box for the action.
2. Click either the up arrow or the down arrow, as needed:

Alternatively, use the Cut , Copy , and Paste  buttons.

To delete an action, click the check box or the action and then click the Delete  button.

4.4 See Also

- [Introduction to the DTL Editor](#)
- [Creating Data Transformations](#)
- [Actions That Set or Clear Values](#)
- [Other Actions](#)

5

DTL Syntax Rules

This topic describes the syntax rules applicable to various DTL actions within [data transformations](#) for [interoperability productions](#).

5.1 References to Message Properties

In most actions within a transformation, it is necessary to refer to properties of the source or target messages. The rules for referring to a property are different depending on the kind of messages you are working with.

- For messages other than virtual documents, use syntax like the following:

```
source.propertyname
```

Or:

```
source.propertyname.subpropertyname
```

Where *propertyname* is a property in the source message, and *subpropertyname* is a property of that property.

If the message includes a collection property, see [Special Variations for Repeating Fields](#). Some of the information there applies to both virtual documents and standard messages.

- For virtual documents other than XML virtual documents, use the syntax described in [Syntax Guide for Virtual Property Paths](#). Also see the following subsection.
- For XML virtual documents, see [Routing XML Virtual Documents in Productions](#).

5.2 Literal Values

When you assign a value to a target property, you often specify a literal value. Literal values are also sometimes suitable in other places, such as the value in a **TRACE** action.

A literal value is either of the following:

- A numeric literal is just a number. For example: 42.3
- A string literal is a set of characters *enclosed by double quotes*. For example: "ABD"

Note: This string cannot include XML reserved characters. For details, see [XML Reserved Characters](#).

For virtual documents, this string cannot include separator characters used by that virtual document format. See [Separator Characters in Virtual Documents](#) and [When XML Reserved Characters Are Also Separators](#).

5.2.1 XML Reserved Characters

Because DTL transformations are saved as XML documents, you must use XML entities in the place of XML reserved characters:

To include this character...	Use this XML entity...
>	>
<	<
&	&
'	'
"	"

For example, to assign the value Joe's "Good Time" Bar & Grill to a target property, set **Value** equal to the following:

```
"Joe&apos;s &quot;Good Time&quot; Bar &amp; Grill"
```

This restriction does not apply inside **CODE** and **SQL** actions, because InterSystems IRIS® automatically wraps a CData block around the text that you enter into the editor. (In the XML standard, a CData block encloses text that should not be parsed as XML. Thus you can include reserved characters in that block.)

5.2.2 Separator Characters in Virtual Documents

In most of the virtual document formats, specific characters are used as separators between segments, between fields, between subfields, and so on. If you need to include any of these characters as literal text when you are setting a value in the message, you must instead use the applicable escape sequence, if any, for that document format.

See the following topics:

- [EDIFACT Separators](#)
- [X12 Separators](#)
- [HL7 Separators](#)
- [ASTM Separators](#)

Important: In a data transformation, the separator characters and escape sequences can be different for the source and target messages. InterSystems IRIS automatically adjusts values as needed, after performing the transformation. This means that you should consider only the separator characters and escape sequences that apply to the *source* message.

5.2.3 When XML Reserved Characters Are Also Separators

- If the character (for example, &) is a separator and you want to include it as a literal character, use the escape sequence that applies to the virtual document format.

- In all other cases, use the XML entity as shown previously in [XML Reserved Characters](#).

5.2.4 Numeric Character Codes

You can include decimal or hexadecimal representations of characters within literal strings.

The string `&#n;` represents a Unicode character when *n* is a decimal Unicode character number. One example is `é` for the Latin e character with acute accent mark (é).

Alternatively, the string `&#xh;` represents a Unicode character when *h* is a hexadecimal Unicode character number. One example is `¿` for the inverted question mark (¿).

5.3 Valid Expressions

When you assign a value to a target property, you can specify an expression, in the language that you selected for the data transformation. You also use expressions in other places, such as the condition for an **IF** action, the value in a **TRACE** action, statements in a **CODE** action, and so on.

The following are all valid expressions:

- Literal values, as described in the [previous section](#).
- Function calls, as described in [Utility Functions for Use in Productions](#). InterSystems IRIS provides a [wizard](#) for these.
- References to properties, as described in [References to Properties](#).
- References to the **aux** variable passed by the rule. If the data transformation is called from a rule, it supplies the following information in the **aux** variable:
 - `aux.BusinessRuleName`—Name of the rule.
 - `aux.RuleReason`—Reason that the rule was fired. It is the same name as used in the logging. An example value is 'rule#1:when#1'. If the RuleReason is longer than 2000 characters, it is truncated to 2000 characters.
 - `aux.RuleUserData`—Value that was assigned in the rule to the property 'RuleUserData'. The value of 'RuleUserData' is always the last value that it was set to.
 - `aux.RuleActionUserData`—Value that was assigned in the rule when or otherwise clause to the property 'RuleActionUserData'.

If the data transformation is called directly from code and not from a rule, the code can pass the auxiliary data in the third parameter. If your data transformation may be called from code that does not set the third parameter, your DTL code should check that the **aux** variable is an object in an **IF** action using the **\$ISOBJECT** function.

- Any expression that combines these, using the syntax of the scripting language you chose for data transformation. See [Specifying Transformation Details](#). Note the following:
 - In ObjectScript, the concatenation operator is the `_` (underscore) character, as in:
`value="prefix"_source.{MSH:ReceivingApplication}_"suffix"`
 - To learn about useful ObjectScript string functions, such as `$CHAR` and `$PIECE`, see the ObjectScript Reference.
 - For a general introduction, see [Using ObjectScript](#).

5.4 See Also

- [Introduction to the DTL Editor](#)
- [Creating Data Transformations](#)
- [Adding and Editing Actions](#)
- [Testing Data Transformations](#)

6

Actions That Set or Clear Values

This topic provides details on setting or clearing values within your DTL [data transformations](#) for [interoperability productions](#). For information on performing these tasks with the legacy UI, see [Assign Actions \(Legacy UI\)](#).

Important: For virtual documents other than XML virtual documents:

- Do not use the **CLEAR**, **APPEND**, or **INSERT**. (To clear a property value in a virtual document, use **SET** with an empty string.)
- Do not manually change escape sequences in the data transformation; InterSystems IRIS® handles these automatically.

6.1 Introduction

There are five kinds of actions that set or clear values: **SET**, **CLEAR**, **REMOVE**, **APPEND**, and **INSERT**.

The DTL diagram shows each of them with a connector line.

6.1.1 Objects and Object References

If you use any of these actions to set a value from the top-level source object or any object property of another object as your source, the target receives a cloned copy of the object rather than the object itself. This prevents inadvertent sharing of object references and saves the effort of generating cloned objects yourself. There is an exception: if the object has a property that is a list or array of objects, only the list reference is cloned, the actual objects within the list retain their original reference, thus still pointing to the source objects.

If you instead want to share object references between source and target, you must **SET** from the source to an intermediate temporary variable, and then **SET** from that variable to the target.

6.2 Adding a SET Action

A **SET** action assigns a value to one or more properties in the target message. To create a **SET** action:

1. [Add an action](#), choosing **SET** from the **New** drop-down list.
2. In the new action, specify the following details:

- **target**—Identifies the property into which the new value will be written. This may be an object property or a virtual document property path. Generally it is a property of the target message used by the transformation. You must enter the target property.

To refer to a property of the target message, type **target** into this field; the editor then displays a menu listing properties of the target message.

- **source**—Specifies the value to use; this can be a property, a literal value, or a more general expression.

To refer to a property of the source message, type **source** into this field; the editor then displays a menu listing properties of the source message. You could also refer to a different property of the target message.

A numeric literal is just a number. For example: 42.3

A string literal is a set of characters *enclosed by double quotes*. For example: "ABD"

Note: This string cannot include XML reserved characters. For virtual documents, this string cannot include separator characters used by that virtual document format. For details, see [Syntax Rules](#).

To create an expression that uses a function, click the Find Functions button . This invokes the [Function Wizard](#).

To create a more complex expression, type the expression into the **source** field. See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

- **key**—Option is relevant only for collection properties; see below.
- **comment**—Specifies an optional description.
- **language**—Select the language for the **source** expression.

6.3 Shortcuts for Adding a SET Action

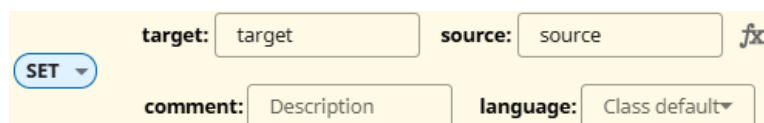
The DTL Editor provides quick ways to add **SET** actions for some simple scenarios.

6.3.1 Copying the Source Message

To create a **SET** action that copies the source message:

1. Click within the **source** box. This box then becomes yellow.
2. Click within the **target** box. This box then becomes yellow.

A connector is drawn between the boxes, and the **Actions** area shows a new **SET** action. The new action looks like this:



6.3.2 Copying a Value from a Source Property to a Target Property

To create a **SET** action that copies a value from a source property to a target property:

1. Click within the box for the source property. This box then becomes yellow.
2. Click within the box for the target property. This box then becomes yellow.

A connector is drawn between the boxes, and the **Actions** area shows a new **SET** action. The new action looks something like this:

6.4 Using the Function Wizard

To access and use the **Function Wizard**:

1. Click the Find Functions button .

2. Select a **Function** from the drop-down list.

More fields display as needed to define the expression.

If you select **Repeat Current Function** from the drop-down list, a copy of the current function is inserted as a parameter of the itself, which creates a recursive call to the function.

3. Edit the fields as needed. For instructions, see the context-sensitive help in the dialog.
4. Click **Save** to save your changes and exit the wizard.

For details on the existing functions, see [Utility Functions for Use in Productions](#). For information on adding custom functions, see [Defining Custom Utility Functions](#).

6.5 SET and Collections

Sometimes a target property is a collection, and you want to set a value within that collection, which could be either of the following kinds of collections:

- Collection properties in standard production messages.
- Repeating fields in XML virtual documents.

To change the value of an item from a collection, [create a SET action](#). For **target**, use syntax that refers to the collection item you want to set. For array properties, use the key of the array item. For list properties, use the index of the list item. For repeating fields in virtual documents, use the index of the segment or field.

For example:

```
target.MyArrayProp("key2")
```

Equivalently, specify **target** so that it omits a reference to the collection item. In this case, specify **key** as the item to change.

6.6 Adding an INSERT Action

This section applies to list properties (but not array properties) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To insert an item into a list:

1. [Add an action](#), choosing **INSERT** from the **New** drop-down list.
2. In the new action, specify the following details:
 - For **target**, select the target list property, for example: `target.MyListProp`
 - Edit **source** to contain a literal value or other valid expression.
See [Valid Expressions](#). Make sure that the expression is valid in the scripting DTL Editor you chose for the data transformation; see [Specifying Transformation Details](#).
 - For **key**, identify the index position for the new item.

For example: 5

6.7 Adding an APPEND Action

This section applies to list properties (but not array properties) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To append an item into a list:

1. [Add an action](#), choosing **APPEND** from the **New** drop-down list.
2. In the new action, specify the following details:
 - For **target**, select the target list property, for example: `target.MyListProp`

- Edit **source** to contain a literal value or other valid expression.

See [Valid Expressions](#). Make sure that the expression is valid in the scripting DTL Editor you chose for the data transformation; see [Specifying Transformation Details](#).

6.8 Adding a REMOVE Action

This section applies to properties in virtual documents.

To remove a property:

1. [Add an action](#), choosing **REMOVE** from the **New** drop-down list.
2. For **target**, select the property to remove.

Important: When you remove properties from a virtual document, it is necessary to perform an additional step known as *building the map* for the message. There are two ways that you can do this:

- Before the steps that remove properties, set the `AutoBuildMap` property to build the map automatically when the properties are deleted. To do this, include a **SET** action that sets `target.AutoBuildMap` equal to 1.
- After the steps that remove properties, call the **BuildMap()** method. To do this, include a **CODE** action that includes this line:

```
do target.BuildMap()
```

6.9 REMOVE and Collections

This section applies to collection properties (lists and arrays) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To remove an item from a collection:

1. [Add an action](#), choosing **REMOVE** from the **New** drop-down list.
2. In the new action, specify the following details:
 - For **target**, select the collection property.
 - For **key**, identify the item to remove.

For array properties, use the key of the array item. For list properties, use the index of the list item. For repeating fields in virtual documents, use the index of the segment or field.

For example:

```
"key2"
```

6.10 Clearing a Collection Property

This section applies to collection properties (lists and arrays) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To clear the contents of a collection:

1. [Add an action](#), choosing **CLEAR** from the **New** drop-down list.
2. For **target**, select the collection property. For example: `target.MyArrayProp`

6.11 See Also

- [Adding and Editing Actions](#)
- [DTL Syntax Rules](#)
- [Other Actions](#)
- [Testing Data Transformations](#)

7

Other Actions

This topic provides details for actions that do not modify values, within a DTL [data transformation](#) for [interoperability productions](#).

For information on performing these tasks with the legacy UI, see [Other Actions \(Legacy UI\)](#).

7.1 Adding an IF Action

An **IF** action executes other actions conditionally, depending on the value of an expression that you provide. InterSystems IRIS® represents each **IF** action as a connector line in the DTL diagram.

To add an **IF** action:

1. [Add an action](#), choosing **IF** from the **New** drop-down list.

The **Actions** area contains two new rows, labeled as follows:

- **IF**—This row marks the beginning of actions to perform if the condition is true.
- **ELSE**—This row marks the beginning of actions to perform if the condition is false.

2. In the **IF** row, edit the **condition** field so that it contains an expression that evaluates to either true or false.

For example:

```
source.ABC = "XYZ"
```

Notes:

- To create an expression that uses a function, click the Find Functions button , select a function, and click **Save**.
- To create a more complex expression, type the expression into the **Value** field. See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

3. To add actions to perform when the condition is true:
 - a. Click the **IF** row.
 - b. Select an item from the **New** drop-down list.
 - c. Edit the new action as needed.

- d. Repeat as necessary.
4. To add actions to perform when the condition is false:
 - a. Click the **ELSE** row.
 - b. Continue as described in the preceding item.

Note: It is not required to have any actions for the **IF** branch or for the **ELSE** branch. If there are no actions in either branch, the **IF** action has no effect.

7.2 Adding a FOR EACH Action

The **FOR EACH** action enables you to define a sequence of actions that is executed iteratively, once for each member of one of the following:

- A collection property (for a standard message).
- A repeating property (for a virtual document).
- A set of subdocuments in a document (for a virtual document).

InterSystems IRIS represents each **FOR EACH** action as a connector line in the DTL diagram.

You can break out of a **FOR EACH** loop at any time by adding a **BREAK** action within the loop.

To add a **FOR EACH** action:

1. [Add an action](#), choosing **FOR EACH** from the **New** drop-down list.
2. For the **target** field, specify a collection or repeating property in the source message.

For the **FOR EACH** action, the **key** field specifies the name of an iterator variable.

The **target** field should not include the iterator key within the parentheses. For example, the following is correct:

```
source.{PID:PatientIdentifierList( )}
```

The **FOR EACH** iterates through the `PatientIdentifierList` repeating fields, starting with the first one (numbered 1) and ending with the last one.

3. The **Unload** check box controls whether to generate code to unload open objects or segments.

If the **Unload** is checked for a **FOR EACH** action, then code is generated in the Transform method to try to unload/unswizzle open object(s) or segment(s) for the property collection at the end of each loop. Unsaved virtual document segments are saved and finalized. If the property is the source object, the source object is usually already saved.

You may still need to manually add actions to unload the target collection's objects or segments. For details on some strategies, see [Unloading Target Collections](#).

The unload of the **FOR EACH** property collection may be unnecessary – for example, for HL7, code generated using CopyValues does not instantiate the source segments.

4. To add actions to the **FOR EACH** block, click the **FOR EACH** action and then add the appropriate actions.

The details are then shown in the block below the DTL diagram.

If the **FOR EACH** applies to a collection property in a message, the sequence of activities is executed iteratively, once for every element that exists within the collection property. If the element is null, the sequence is not executed. The sequence is executed if the element has an empty value, that is, the separators are there but there is no value between them, but is not executed for a null value, that is, the message is terminated before the field is specified.

7.2.1 Shortcuts for the FOR EACH Action

When you are working with virtual documents, InterSystems IRIS provides a shortcut notation that iterates through every instance of a repeating field within a document structure. This means you do not actually need to set up multiple nested **FOR EACH** loops to handle repeating fields; instead you create a single **assign** action using a virtual property path with empty parentheses within the curly bracket { } syntax. For information, see [Curly Bracket {} Syntax](#).

Note: If the source and target types are different, you cannot use this shortcut for the **FOR EACH** action. Use an explicit **FOR EACH** action in these cases.

7.2.2 Unloading Target Collections

While the **Unload** option automatically removes objects from a source collection, you need to add custom code at the end of a **FOR EACH** action to remove objects from a target collection. In a simple example in which the target is a complex record, you could use the following code to save the current target record and then unload it:

```
Do target.Record16.GetAt(k1).%Save(0)
Do target.Record16.%UnSwizzleAt(k1)
```

In other scenarios, it might be better to avoid loading the target altogether in order to avoid issues where the target is not unloaded. For example, suppose you have an object that has a parent/child property with many children. Within the **FOR EACH** action, you have a subtransform combined with *propSetObjectId(parentId)*, where *prop* is the name of the property.

In this example, the target is the batch object, the target class is `Demo.RecordMapBatch.Map.TrainDataOut.BatchOut` and the record class is `Demo.RecordMapBatch.Transform.Optimized.Record`

Before your **FOR EACH** loop, you need to create an empty target and assign its ID to a property `BatchOutID`:

```
<assign value='target.%Save()' property='tsc' action='set' />
<assign value='target.%Id()' property='BatchOutID' action='set' />
<assign value='target' property='' action='set' />
```

Then, in the **FOR EACH** loop, you can use code that directly impacts the target without having the target instantiated. For example:

```
<assign value='' property='record' action='set' />
<subtransform class='Demo.RecordMapBatch.Transform.Optimized.Record' targetObj='record'
sourceObj='source.Records.(k1)' />

<comment>
<annotation>Assign record to target directly. </annotation>
</comment>
<assign value='record.%ParentBatchSetObjectId(BatchOutID)' property='tsc' action='set' />
<assign value='record.%Save()' property='tsc' action='set' />
```

Then, before the DTL ends, set the variable `target` back to the expected product of the DTL. For example:

```
<assign value='##class(Demo.RecordMapBatch.Map.TrainDataOut.BatchOut).%OpenId(BatchOutID)'
property='target' action='set' />
```

7.2.3 Avoiding <STORE> Errors with Large Messages

As you loop over segments in messages or object collections, they are brought into memory. If these objects consume all the memory assigned to the current process, you may get unexpected errors. You can avoid these errors in the source collection by using the **Unload** option in the Management Portal. For some strategies for removing objects in a target collection, see [Unloading Target Collections](#).

As another strategy, if you are processing many segments in a **FOR EACH** loop, you can call the **commitSegmentByPath()** method on both the source and target as the last step in the loop. Similarly, for object collections, use the **%UnSwizzleAt()** method.

The method **commitCollectionOpenSegments()** loops through the runtimePath looking for open segments within the specified collection path and calls **commitSegmentByPath()** for each open segment. This method is available from the classes `EnsLib.EDI.X12.Document`, `EnsLib.EDI.ASTM.Document`, `EnsLib.EDI.EDIFACT.Document`, and `EnsLib.HL7.Message`.

If you cannot make code changes, a temporary workaround is to increase the amount of memory allocated for each process. You can change this by setting the *bbsiz* parameter on the **Advanced Memory Settings** page in the Management Portal. Note that this action requires a system restart, and you should consult with your system administrator before performing it.

7.3 Adding a SUBTRANSFORM Action

A **SUBTRANSFORM** action invokes another transformation (an ordinary transformation), often within a **FOR EACH** loop. Subtransformations are particularly useful with virtual documents, because EDI formats are typically based on a set of segments that are used in many message types. The ability to reuse a transformation within another transformation means that you can create a reusable library of segment transformations that you can call as needed, without duplicating code transformation.

InterSystems IRIS does not represent a **SUBTRANSFORM** action in the DTL diagram.

To add a **SUBTRANSFORM** action:

1. [Add an action](#), choosing **SUBTRANSFORM** from the **New** drop-down list.
2. In the new action, specify the following details:
 - **target**—Identifies the property into which the transformed value will be written. This may be an object property or a virtual document property path. Generally it is a property of the target message used by the transformation. You must enter the target property.
 - **source**—Identifies the property being transformed. This may be an object property or a virtual document property path. Generally it is a property of the source message used by the transformation. You must enter the source property.
 - **auxiliary property**—Optionally, specifies a value to be passed to the subtransform. The subtransform accesses the value as the aux variable. To pass multiple values:
 - a. Create an array variable with subscripts as in the following example:

```
set MyVar(1)="first value"
set MyVar(2)="second value"
```
 - b. Include a period immediately before the name of this variable, within the **auxiliary property** field. (The period indicates that this variable is passed by reference, which is the required way to pass a variable that has subscripts.)

Within the subtransform, you can access these values as `aux(1)` and `aux(2)`. That is, the *aux* variable has the same subscripts that you specified in the input array variable.

- **class**—Specifies the data transformation class to use. This can be either a DTL transformation or a custom transformation. For information on custom transformations, see [Defining Custom Transformations](#). You must enter the class.
- **comment**—Specifies an optional comment.

Note: In the case of a **SUBTRANSFORM** with **Mode** as **Create new** or **Copy**, it is not necessary to have a pre-existing target object.

7.4 Adding a TRACE Action

A **TRACE** action generates a trace message, which is helpful for diagnosis. If the [Log Trace Events](#) setting is enabled for the parent business host, this message is written to the Event Log. If the [Foreground](#) setting is enabled for the parent business host, the trace messages are *also* written to the Terminal window.

InterSystems IRIS does not represent a **TRACE** action in the DTL diagram.

To add a **TRACE** action:

1. [Add an action](#), choosing **TRACE** from the **New** drop-down list.
2. In the new action, specify the following:
 - **source**—Specify a literal value or other valid expression.
See [Valid Expressions](#). Make sure that the expression is valid in the selected language.
 - **comment**—Specify an optional description.
 - **language**—Select the language for this expression.

The **TRACE** action generates trace message with User priority; the result is the same as using the `$$$TRACE` macro in ObjectScript.

7.5 Adding a CODE Action

A **CODE** action enables you to execute one or more lines of user-written code within a DTL data transformation. This option enables you to perform tasks that are difficult to express using the DTL elements. InterSystems IRIS does not represent a **CODE** action in the DTL diagram.

To add a **CODE** action:

1. [Add an action](#), choosing **CODE** from the **New** drop-down list.
2. In the new action, specify the following:
 - **code**—Specify one or more lines of code in the specified language. For rules about expressions in this code, see [Syntax Rules](#).

If you are using ObjectScript, make sure that each line starts with a space.

InterSystems IRIS automatically wraps your code within a CDATA block. This means that you do not have to escape special XML characters such as the apostrophe (') or the ampersand (&),

Also see the notes below.

- **comment**—Specify an optional description.
- **language**—Select the language for this expression.

Tip: To write custom code that you can debug easily, write the code within a class method or a routine so that it can be executed in the Terminal. Debug the code there. Then call the method or routine from within the code action of the DTL.

7.5.1 Guidelines for Using Custom Code in DTL

In order to ensure that execution of a data transformation can be suspended and restored, you should follow these guidelines when using a code action:

- The execution time should be short; custom code should not tie up the general execution of the data transformation.
- Do not allocate any system resources (such as taking out locks or opening devices) without releasing them within the same code action.
- If a code action starts a transaction, make sure that the same action ends the transactions in *all possible scenarios*; otherwise, the transaction can be left open indefinitely. This could prevent other processing or can cause significant downtime.

If you are using ObjectScript, make sure that each line starts with a space.

7.6 Adding an SQL Action

An **SQL** action enables you to execute an SQL SELECT statement from within the DTL transformation. InterSystems IRIS does not represent an **SQL** action in the DTL diagram.

To add an **SQL** action:

1. [Add an action](#), choosing **SQL** from the **New** drop-down list.
2. In the new action, specify the following:

- **code**—Specify a valid SQL SELECT statement.

InterSystems IRIS automatically wraps your SQL within a CDATA block. This means that you do not have to escape special XML characters such as the apostrophe (') or the ampersand (&).

Also see the notes below.

- **comment**—Specify an optional description.

7.6.1 Guidelines for Using SQL in DTL

Be sure to use the following guidelines:

- Always use the fully qualified name of the table, including both the SQL schema name and table name, as in:

`MyApp.PatientTable`

Where `MyApp` is the SQL schema name and `PatientTable` is the table name.

- Any tables listed in the FROM clause must either be stored within the local InterSystems IRIS database or linked to an external relational database using the SQL Gateway.
- Within the INTO and WHERE clauses of the SQL query, you can refer to a property of the source or target object. To do so, place a colon (:) in front of the property name. For example:

```
SELECT Name INTO :target.Name FROM MainFrame.EmployeeRecord WHERE SSN = :source.SSN AND City = :source.Home.City
```

- Only the first row returned by the query will be used. Make sure that the WHERE clause correctly specifies the desired row.

7.7 Adding a SWITCH Action

A **SWITCH** action contains a sequence of one or more **CASE** actions and a **DEFAULT** action. When a **SWITCH** action is executed, it begins evaluating each **CASE** condition. When an expression evaluates to true, then the contents of the corresponding **CASE** block are executed; otherwise, the expression for the next **CASE** action is evaluated. As soon as one of the **CASE** actions is executed, the execution path of the transformation leaves the **SWITCH** block without evaluating any other conditions. If no **CASE** condition is true, the contents of the **DEFAULT** action are executed and then control leaves the **SWITCH** block.

To add an **SWITCH** action:

1. [Add an action](#), choosing **SWITCH** from the **New** drop-down list.

This adds three rows to the **Actions** area, labeled **SWITCH**, **CASE**, and **DEFAULT**.

2. In the **SWITCH** row, specify the following:
 - comment**—Specify an optional description.
 - language**—Select the language for the expressions used in this action.
3. Add more **CASE** actions if needed.
4. Modify the **CASE** rows as follows:
 - condition**, specify the condition. You can click the magnifying glass to add a function as part of the condition.
 - comment**—Specify an optional description.
5. Optionally modify the **DEFAULT** row. It is not necessary to include any steps within the **DEFAULT** action.
6. In each of these branches, add actions to perform in the given scenarios. For example, you may want to set a target property a specific way when a condition is true.

7.8 Adding a CASE Action

Use the **CASE** action within a **SWITCH** block to execute a block of actions when a condition is matched. When a **CASE** condition is met and the block of actions performed, the execution path of the transformation leaves the **SWITCH** block without evaluating any other conditions.

To add a **CASE** action:

1. Select a **SWITCH** action in the **Actions** area.
2. Select **CASE** from the **New** drop-down list.
3. For **condition**, specify the condition. You can click the magnifying glass to add a function as part of the condition.
4. With the **CASE** action selected in the **Actions** area, use the **New** drop-down to add the actions that will be executed if the condition evaluates to true.

7.9 Adding a Default Action

You cannot add a **DEFAULT** block by using the **New** drop-down list. Rather, the **DEFAULT** action is automatically added to a **SWITCH** block when you add the **SWITCH** action. The actions contained in the **DEFAULT** block are executed if none of the **CASE** conditions in the **SWITCH** block are met. If you do not want anything to happen when none of the **CASE** conditions are met, simply leave the **DEFAULT** block empty.

7.10 Adding a Break Action

Add a **BREAK** action to a **FOR EACH** loop to leave the loop as soon as the **BREAK** action is executed. After the **BREAK** action is executed, the data transformation continues to process the action immediately following the **FOR EACH** loop.

If you add a **BREAK** action outside of a **FOR EACH** loop, the data transformation terminates as soon as the **BREAK** action is executed.

7.11 Adding a COMMENT Action

To help annotate the actions in a data transformation, you can add a comment that appears in the list of actions. After selecting **Add Action > Comment**, enter the comment in the **Description** text box in the **Actions** area.

7.12 See Also

- [Adding and Editing Actions](#)
- [DTL Syntax Rules](#)
- [Actions That Set or Clear Values](#)
- [Testing Data Transformations](#)

8

Listing and Managing Data Transformations

The **Data Transformation List** page enables you to list, import, export, test, and delete [data transformations](#), which are a form of [business logic](#) you can use within [interoperability productions](#).

8.1 Introduction

To access the **Data Transformation List** page in the Management Portal, click **Interoperability > List > Data Transformations**.

The page lists the data transformation classes defined in the current namespace. This page lists two kinds of transformations:

- DTL transformations are displayed in blue. You can double-click one to open it in the [DTL Editor](#).
- Custom transformations are displayed in black. These classes are based on `Ens.DataTransform` and do not use DTL. You must edit these in [your IDE](#).

8.2 Options on This Page

To use this page, select a data transformation and then click one of the following commands in the ribbon bar:

- **Edit**—(*DTL transformations only*) Click to change or view the data transformation using the [DTL Editor](#).
- **Test**—Click to test the selected transformation class using the Test Transform wizard.

For details, see [Testing Data Transformations](#).

- **Delete**—Click to delete the selected transformation class.
- **Export**—Click to export the selected transformation class to an XML file.
- **Import**—Click to import a data transformation that was exported to an XML file.

8.3 Related Options

You can also export and import these classes as you do any other class in InterSystems IRIS. You can use the **System Explorer** > **Globals** page of the Management Portal.

8.4 See Also

- [Comparison of Business Logic Tools](#)
- [Introduction to DTL Tools](#)
- [Introduction to the DTL Editor](#)
- [Creating Data Transformations](#)
- [Testing Data Transformations](#)

9

Testing Data Transformations

After you compile a [data transformation](#) class, you can (and should) test it. This topic describes how to do so.

Note: This topic applies to both DTL transformations and custom transformations.

9.1 Using the Transformation Testing Page

The Management Portal provides the **Test Transform** wizard. You can access this from the following locations in the Management Portal:

- Click **Test** from the **Tools** tab in the Data Transformation Builder
- Select the transformation and click **Test** on the Data Transformation List page.

Initially the **Output Message** window is blank and the **Input Message** window contains a text skeleton in a format appropriate to the source message. To test:

1. If your DTL code references the properties of the aux, context, or process systems objects, enter values for these properties to see the results as if the data transformation was invoked with these objects instantiated. The table for entering values appears only if the DTL references the internal properties of aux, process, or context systems objects.
2. Edit the **Input Message** so that it contains appropriate data. What displays and what you enter in the input box depends on your source type and class:
 - For EDI messages, the window displays raw text; have some saved text files ready so that you can copy and paste text from these files into the **Input Message** box.
 - For regular production messages, the window displays an XML skeleton with an entry for each of the properties in the message object; type in a value for each property.
 - For record maps, complex record maps, and batch record maps, you can enter raw text or XML.
3. Click **Test**.
4. Review the results in the **Output Message** box.

9.2 Testing a Transformation Programmatically

To test a transformation programmatically, do the following in the Terminal (or write a routine or class method that contains these steps):

1. Create an instance of the source message class.
2. Set properties of that instance.
3. Invoke the **Transform()** class method of your transformation class. This method has the following signature:

```
classmethod Transform(source As %RegisteredObject, ByRef target As %RegisteredObject) as %Status
```

Where:

- *source* is the source message.
 - *target* is the target message created by the transformation.
4. Examine the target message and see if it has been transformed as wanted. For an easy way to examine both messages in XML format, do the following:
 - a. Create an instance of %XML.Writer.
 - b. Optionally set the Indent property of that instance equal to 1.
This adds line breaks to the output.
 - c. Call the **RootObject()** method of the writer instance, passing the source message as the argument.
 - d. Kill the writer instance.
 - e. Repeat with the target message.

For example:

ObjectScript

```
//create an instance of the source message
set source=##class(DTLTest.Message).CreateOne()
set writer=##class(%XML.Writer).%New()
set writer.Indent=1 do writer.RootObject(source)
write !!
set sc=##class(DTLTest.Xform1).Transform(source, .target)
if $$$ISERR(sc)
  {do $system.Status.DisplayError(sc)}
set writer=##class(%XML.Writer).%New()
set writer.Indent=1
do writer.RootObject(target)
```

9.3 See Also

- [Introduction to DTL Tools](#)
- [Introduction to the DTL Editor](#)
- [Creating Data Transformations](#)
- [DTL Syntax Rules](#)
- [Listing and Managing Data Transformations](#)

DTL Reference

This reference provides detailed information about each DTL element.

DTL <annotation>

Provides a descriptive comment for a DTL element, within a [DTL transformation](#).

Syntax

```
<annotation>
  <![CDATA[ Sends patient data from lab to CRM system. ]]>
</annotation>
```

Description

The <annotation> element allows you to associate a descriptive comment with a DTL element. An <annotation> must appear as the first child of the element that it is annotating. For example:

XML

```
<transform targetClass='Demo.DTL.ExampleTarget'
  sourceClass='Demo.DTL.ExampleSource'
  create='new'
  language='objectscript'>
  <annotation>
    <![CDATA[Implement current naming conventions.]]>
  </annotation>
  <trace value="Convert from lowercase to uppercase"/>
  <assign property='target.UpperCase'
    value='$ZCONVERT(source.LowerCase,"U")'
    action='set'>
    <annotation>This is a comment for the assign element</annotation>
  </assign>
</transform>
```

The previous example uses CDATA syntax around the annotation text. This convention is optional, but it lets you use line breaks and special characters such as the apostrophe (') without worrying about XML escape sequences. The maximum length of the <annotation> string is 32,767 characters, including the CDATA escape characters.

Also notice that the annotation for the assign element appears as a child immediately following the opening assign tag.

Most elements within DTL support [<annotation>](#) as a child element. This allows you to associate a descriptive comment with a DTL element. Unlike BPL, which offers positional attributes for every element, <annotation> is the *only* child element or attribute that most DTL elements have in common. If you use the <annotation> element, it must appear as the first child of the element that it is annotating.

DTL <assign>

Assigns a value to a property of an object, within a [DTL transformation](#).

Syntax

```
<assign property="propertyname" value="expression" />
```

Attributes

Attribute	Description	Value
<i>property</i>	Required. The property that is the target of this assignment.	A string.
<i>value</i>	Required. Provides a value for the property.	An ObjectScript expression that provides a valid value for the property.
<i>action</i>	Optional. If <i>value</i> is a collection property (list or array), then use <i>action</i> to specify the type of assignment to perform. The default is a set action.	One of the following values: set, clear, remove, append, insert. See the actions section for details.
<i>key</i>	Optional, except in some cases when <i>value</i> is a collection property (list or array). If so, then use this key to specify the element upon which the assignment will be performed.	A string that is an expression that evaluates to a key.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <assign> element.

Description

The DTL <assign> element is used from within a DTL [<transform>](#) element to specify a target property and an expression whose value will be assigned to it. Generally, this expression involves values from the source object for the data transformation, but they may also be literal values. All properties involved in a DTL <assign> activity must be properties within the source or target object for the data transformation.

The source and target objects are generally production message body objects, as described in [Messages](#). These consist of a message header and a message body object.

Properties in the standard production message body can be data types, objects, or collections of either. Collection properties are declared with either [Collection = list] or [Collection = array] in the class definition. You can refer to the properties on the standard production message body using dot syntax as for any object property.

Properties in a virtual document require the unique syntax described in the following topics:

- [Virtual Property Paths](#)
- [Syntax Guide for Virtual Property Paths](#)

Actions of the <assign> Element

There are several types of DTL <assign> operation, as specified by the optional action attribute. Aside from the default of *set*, these variations are intended to handle assignments involving collection properties within a standard production message body. The following table describes the actions of the <assign> element.

Assign action	Description	Example
set	Sets the value of the specified property to that of the value attribute. Note that the value attribute contains an expression and can itself refer to an object or property of an object.	The following statement sets the value of the target BankName property: XML <pre><assign property='target.BankName' value='process.BankName' action='set' /></pre>
append	Adds the target element to the end of a list property	
clear	Clears the contents of the specified collection property. The value and key attributes are ignored. (Applies to collection properties only.)	The following statement clears the contents of the collection property List: XML <pre><assign property='target.List' action='clear' /></pre>
insert	Inserts a value into the specified collection property. If the key attribute is present the new value is inserted after the position (an integer) specified by key; otherwise, the new item is inserted at the end. (Applies to list collection properties only.)	The following statement inserts a value into the array collection property Array using the key primary: XML <pre><assign property='target.Array' action='insert' key='primary' value='source.Primary' /></pre>
remove	Removes an item from the specified collection property. The value attribute is ignored. (Applies to collection properties only.)	

Note: Virtual documents do not use any action value other than *set* or *remove*.

The **set** action sets the value of the specified property to that of the value attribute. Note that the value attribute contains an expression and can itself refer to an object or property of an object:

XML

```
<assign property='target.SSN' value='source.SSN' />
```

If the target property is an array collection, then the value of the key attribute specifies an item in the array, otherwise the key attribute is ignored.

If the target property is a collection and the value attribute specifies a collection of the same type, then the collection contents are copied into the target collection:

XML

```
<assign property='target.List' value='source.List' />
```

The default action for the assign element is the set operation; if action is not specified, then the assign specifies a set operation.

Objects and Object References

If you <assign> from the top-level source object or any object property of another object as your source, the target receives a cloned copy of the object rather than the object itself. This prevents inadvertent sharing of object references and saves the effort of generating cloned objects yourself. However, if you want to share object references between source and target you must <assign> from the source to an intermediate temporary variable, and then <assign> from that variable to the target.

Wholesale Copy

To create a target object that is an exact copy of the source, do not use:

```
<assign property='target' value='source' />
```

Instead use the `create='copy'` attribute in the containing `<transform>` element.

The `create` option may have one of the following values:

- `new`—Create a new object of the target type, before executing the elements within the data transformation. This is the default.
- `copy`—Create a copy of the source object to use as the target object, before executing the elements within the transform.
- `existing`—Use an existing object, provided by the caller of the data transformation, as the target object.

DTL <break>

Terminates a <foreach> loop or stop processing a [DTL transformation](#).

Syntax

```
<break/>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <break> element.

Description

When included in a <foreach> element, the <break> element terminates the For Each loop. If <break> is outside of a For Each loop, the entire data transformation terminates as soon as the break is executed.

DTL <case>

Executes a block of actions within a <switch> element when the specified condition is met, within a [DTL transformation](#).

Syntax

```
<switch>
  <case condition="1">
    ...
  </case>
  <default>
    ...
  </default>
</switch>
```

Attributes

Attribute	Description	Value
<i>condition</i>	Required. An ObjectScript expression that, if true, causes the contents of the <case> element to be executed.	An expression that evaluates to the integer value 1 (if true) or 0 (if false).

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <case> element.

Description

The <switch> element contains one or more <case> elements. The elements within a <case> element are executed if the condition evaluates to true.

DTL `<code>`

Executes one or more lines of custom code, within a [DTL transformation](#).

Syntax

```
<code>
  <![CDATA[ target.Name = source.FirstName & " " & source.LastName]]>
</code>
```

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <code><code></code> element.

Description

The DTL `<code>` element executes one or more lines of user-written code within a DTL data transformation. You can use the `<code>` element to perform special tasks that are difficult to express using the DTL elements. Any properties referenced by the `<code>` element must be properties within the source or target object for the data transformation.

The scripting language for a DTL `<code>` element is specified by the `language` attribute of the containing [<transform>](#) element. The value should be `objectscript`. Any expressions found in the data transformation, as well as lines of code within `<code>` elements, must use the specified language.

For further information, see the following items:

- Using ObjectScript
- ObjectScript Reference

Typically a developer wraps the contents of a `<code>` element within a CDATA block to avoid having to worry about escaping special XML characters such as the apostrophe (') or the ampersand (&). For example:

XML

```
<code>
  <![CDATA[ target.Name = source.FirstName & " " & source.LastName]]>
</code>
```

In order to ensure that execution of a data transformation can be suspended and restored, you should follow these guidelines when using the `<code>` element:

- The execution time should be short; custom code should not tie up the general execution of the data transformation.
- Do not allocate any system resources (such as taking out locks or opening devices) without releasing them within the same `<code>` element.
- If a `<code>` element starts a transaction, make sure that the same `<code>` element ends the transactions in *all possible scenarios*; otherwise, the transaction can be left open indefinitely. This could prevent other processing or can cause significant downtime.

Available Variables

The variables that are available in a DTL `<code>` element are dependent upon the method used to call the element. Refer to the following table to see the available variables and their properties:

Variable Name	Purpose	Available when DTL is called through:
<i>source</i>	Contains properties of the source message.	All methods
<i>target</i>	Contains properties of the target message.	All methods
<i>process</i>	The <i>process</i> object represents the current instance of the BPL business process object (an instance of the BPL class). This object has one property for each property defined in that class. You can invoke methods of the <i>process</i> object; for example: <code>process . SendRequestSync ()</code>	BPL business processes
<i>context</i>	The <i>context</i> object is a general-purpose data container for the business process. <i>context</i> has no automatic definition. To define properties of this object, use the <context> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: <code>context . Balance</code>	BPL business processes
<i>aux</i>	Contains information from the business rule that called the DTL.	Business rules

DTL <comment>

Adds comments to a [DTL transformation](#).

Syntax

```
<comment>
  <annotation>
    ...
  </annotation>
</comment>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	A text string that contains the comment.

Description

The contents of the <annotation> element of <comment> appear in the Management Portal to describe the DTL actions.

DTL <default>

Executes contents if none of the <case> elements in a <switch> element evaluate to true, within a [DTL transformation](#).

Syntax

```
<switch>
  <case condition="1">
    ...
  </case>
  <default>
    ...
  </default>
</switch>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <default> element.

Description

The <default> element appears at the end of the <switch> element, and is executed if none of the <case> elements evaluate to true.

DTL <false>

Performs a set of activities when the condition for an <if> element is false, within a [DTL transformation](#).

Syntax

```
<if condition="0">
  <true>
    ...
  </true>
  <false>
    ...
  </false>
</if>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <false> element.
Most activities	Optional. <false> may contain zero or more of the following elements in any combination: <assign>, <code>, <foreach>, <if>, <sql>, <subtransform>, or <trace>.

Description

A <false> element is used within an [<if>](#) to contain elements that need to be executed if the condition is false.

DTL <foreach>

Defines a sequence of activities to be executed iteratively, within a [DTL transformation](#).

Syntax

```
<foreach property="P1" key="K1">
    .
    .
    .
</foreach>
```

Attributes

Attribute	Description	Value
<i>property</i>	Required. The collection property (list or array) to iterate over. It must be the name of a valid object and property in the execution context.	A string of one or more characters.
<i>key</i>	Required. The index used to iterate through the collection. It must be a name of a valid object and property in the execution context. It is assigned a value for each element in the collection.	A string of one or more characters.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <foreach> element.
Most activities	Optional. <foreach> may contain zero or more of the following elements in any combination: <assign>, <code>, <foreach>, <if>, <sql>, <subtransform>, or <trace>.

Description

The <foreach> element defines a sequence of activities that are executed iteratively, once for every element that exists within a specified collection property. If the element is null, the sequence is not executed. The sequence is executed if the element has an empty value, that is, the separators are there but there is no value between them, but is not executed for a null value, that is, the message is terminated before the field is specified.

For example:

XML

```
<foreach key='i' property='target.{PID:3()}'>
    <assign property='target.{PID:3(i).4}' value='"001"' action='set' />
</foreach>
```

Or:

XML

```
<foreach key='key' property='source.{PID:PatientIDInternalID()}'>
  <if condition='source.{PID:PatientIDInternalID(key).identifiertypecode}="PAS"'>
    <true>
      <assign property='target.{PID:PatientIdentifierList(key).identifiertypecode}'
              value='"MR"'
              action='set' />
    </true>
  </if>
  <if condition='source.{PID:PatientIDInternalID(key).identifiertypecode}="GMS"'>
    <true>
      <assign property='target.{PID:PatientIdentifierList(key).identifiertypecode}'
              value='"MC"'
              action='set' />
      <assign property='target.{PID:PatientIdentifierList(key).assigningfacility}'
              value='"AUSHIC"'
              action='set' />
    </true>
  </if>
</foreach>
```

The properties referenced by the `<foreach>` element must be properties in the source or target object for the data transformation.

Nested `<foreach>`

Nesting of `<foreach>` elements is allowed, but see the next subsection for an alternative.

Shortcuts for `<foreach>`

When you are working with a document-based message or “virtual document” type, the `<assign>` statement offers a shortcut notation that iterates through every instance of a repeating field within a document structure. This means you do not actually need to set up `<foreach>` loops with 'i' 'j' and 'k' just for the purpose of handling repeating fields. Instead, you can use a much simpler notation with empty parentheses. See [Iterating Through Repeating Fields](#).

Avoiding `<STORE>` Errors with Large Messages

As you loop over segments in a message or object collections, they are brought into memory. If these objects consume all the memory assigned to the current process, you may get unexpected errors.

To avoid this, remove the objects from memory after you no longer need them. For example, if you are processing many segments in a `<foreach>` loop, you can call the `commitSegmentByPath` method on both the source and target as the last step in the loop. Similarly, for object collections, use the `%UnSwizzleAt` method.

If you cannot make code changes, a temporary workaround is to increase the amount of memory allocated for each process. You can change this by setting the `bbsiz` parameter on the **Advanced Memory Settings** page in the Management Portal. Note that this requires a system restart and should only occur after consulting with your system administrator.

DTL <group>

Organizes related elements into a display unit, within a [DTL transformation](#).

Syntax

```
<group>  
...  
</group>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <group> element.
All	Any element can be added to a <group> element.

Description

The <group> element organizes related elements into a logical unit.

DTL <if>

Evaluates a condition and performs one action if true, another if false, within a [DTL transformation](#).

Syntax

```
<if condition="1">
  <true>
    ...
  </true>
  <false>
    ...
  </false>
</if>
```

Attributes

Attribute	Description	Value
<i>condition</i>	Required. An ObjectScript expression that, if true, causes the contents of the <true> element to execute. If false, the contents of the <false> element are executed.	An expression that evaluates to the integer value 1 (if true) or 0 (if false).

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <if> element.
<true>	Optional. If the condition is true, activities inside the <true> element are executed.
<false>	Optional. If the condition is false, activities inside the <false> element are executed.

Description

The <if> element evaluates an expression and, depending on its value, executes one of two sets of activities (one if the expression evaluates to a true value, the other if it evaluates to a false value).

The <if> element may contain a <true> element and a <false> element which define the actions to execute if the expression evaluates to true or false, respectively.

If both <true> and <false> elements are provided, they may appear within the <if> element in any order.

If the condition is true and there is no <true> element, or if the condition is false and there is no <false> element, no activity results from the <if> element.

DTL <sql>

Executes an embedded SQL SELECT statement within a data transformation, within a [DTL transformation](#).

Syntax

```
<sql>
  <![CDATA[
    SELECT SSN INTO :context.SSN
    FROM MyApp.PatientTable
    WHERE PatID = :request.PatID ]]>
</sql>
```

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <sql> element.

Description

The DTL <sql> element executes an arbitrary embedded SQL SELECT statement from within a DTL [<transform>](#) element.

To use the <sql> element effectively, keep the following tips in mind:

- Always use the fully qualified name of the table, including both the SQL schema name and table name, as in:
MyApp.PatientTable
 Where **MyApp** is the SQL schema name and **PatientTable** is the table name.
- The contents of the <sql> element must contain a valid embedded SQL SELECT statement.
 It is convenient to place the SQL query within a CDATA block so that you do not have to worry about escaping special XML characters.
- Any tables listed in the SQL query's FROM clause must either be stored within the local InterSystems IRIS database or linked to an external relational database using the SQL Gateway.
- Within the INTO and WHERE clauses of the SQL query, you can refer to a property of the source or target object by placing a : (colon) in front of the variable name. For example:

XML

```
<sql><![CDATA[
  SELECT Name INTO :target.Name
  FROM MainFrame.EmployeeRecord
  WHERE SSN = :source.SSN AND City = :source.Home.City
]]>
</sql>
```

- Only the first row returned by the query will be used. Make sure that your WHERE clause correctly specifies the desired row.

DTL <subtransform>

Invokes another data transformation, within a [DTL transformation](#).

Syntax

```
<subtransform class='class-name'
               targetObj='target-value'
               sourceObj='source-value' />
```

Attributes

Attribute	Description	Value
<i>class</i>	Required. Name of the class that contains the data transformation to be invoked. This class must be in the same namespace as the class that invokes it. Often, <i>class</i> is a DTL data transformation defined using a DTL <transform> element, as shown in the examples in this topic. Alternatively, <i>class</i> can identify a custom subclass of <code>Ens.DataTransform</code> that implements the Transform method and does not use DTL.	The full package and class name.
<i>sourceObject</i>	Required. Identifies the property being transformed. This may be an object property or a virtual document property. Generally it is a property of the source object identified by the containing <transform> element's <i>sourceClass</i> and (for virtual documents) <i>sourceDocType</i>. In this case it is referenced using dot syntax as follows: <code>source.property</code> or <code>source.{propertyPath}</code>	Property name. For virtual documents and their segments, use virtual property syntax.
<i>targetObject</i>	Required. Identifies the property into which the transformed value will be written. This may be an object property or a virtual document property. Generally it is a property of the target object identified by the containing <transform> element's <i>targetClass</i> and (for virtual documents) <i>targetDocType</i>. In this case it is referenced using dot syntax as follows: <code>target.property</code> or <code>target.{propertyPath}</code> In the case of a subtransform with Create as <code>new</code> or <code>copy</code>, it is not necessary to have a pre-existing target object.	Property name. For virtual documents and their segments, use virtual property syntax.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <subtransform> element.

Description

The <subtransform> element invokes another data transformation. Making a call to <subtransform> allows the containing <transform> element to invoke other data transformations to complete segments of its work. This allows developers greater flexibility in maintaining a suite of reusable DTL transformation code.

Before the <subtransform> element was available, every DTL <transform> stood alone. In order to write multiple DTL transformations that contained an identical sequence of actions, it was necessary to copy and paste the corresponding sections of code from one class into another. Now, each of these DTL classes can replace repeated lines with a <subtransform> element that invokes another class to perform the desired sequence.

The source or target objects for a <subtransform> may be ordinary InterSystems IRIS objects, virtual document *message* objects, or virtual document *segment* objects representing an individual segment within a virtual document message. The <subtransform> is especially important for interface developers working with Electronic Data Interchange (EDI) formats, where each message or document may contain many independent segments that need to be transformed. Having the <subtransform> available means you can create a reusable library of segment transformations that you can call as needed, without duplicating code in the calling transformation.

For virtual documents and their segments, you must use virtual property syntax, such as the {} curly bracket syntax in the following examples. The property path inside the brackets must refer to a particular segment, not to a field within a segment or to a group of segments. For background information, see [Using Virtual Documents in Productions](#); details are available in [Virtual Property Path](#).

DTL <switch>

Evaluates <case> elements and executes the contents of the first one that evaluates to true, within a [DTL transformation](#).

Syntax

```
<switch>
  <case condition="1">
    ...
  </case>
  <default>
    ...
  </default>
</switch>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <switch> element.
<case>	The first <case> element that evaluates to true is executed.
<default>	Optional. If none of the <case> elements evaluate to true, the contents of the <default> element are executed.

Description

The <switch> element contains one or more <case> elements along with an optional <default> element. The contents of a <case> element are executed if the condition evaluates to true. Once a <case> element evaluates to true, none of the other <case> elements nor the <default> element are evaluated. The contents of the <default> element are executed if none of the <case> elements evaluate to true.

DTL <trace>

Writes a message to the foreground ObjectScript shell, within a [DTL transformation](#).

Syntax

```
<trace value='\"The time is: \"_$ZDATETIME($H,3)\"' />
```

Attributes

Attribute	Description	Value
<i>value</i>	Required. This is the text for the trace message. It can be a literal text string or an ObjectScript expression to be evaluated.	A string of one or more characters. May be a literal string or an expression.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <trace> element.

Description

The <trace> element writes a message to the ObjectScript shell. <trace> messages appear only if the business host that invokes the DTL data transformation has been configured to **Run in Foreground** mode.

Trace messages may be written to the [Event Log](#) as well as to the console. A system administrator controls this behavior from the Management Portal Configuration page. If the business host that invokes the DTL data transformation has the **Log Trace Events** option checked, it writes trace messages to the [Event Log](#) as well as displaying them at the console. If a trace message is logged, its [Event Log](#) entry type is Trace.

The DTL <trace> element generates trace message with User priority; the result is the same as calling the \$\$\$TRACE utility from ObjectScript.

Note: For details, see [Adding Trace Elements](#).

DTL <transform>

Defines a [DTL transformation](#).

Syntax

```
<transform sourceClass="MyApp.SAPtoJDE"
          targetClass="AlsoMine.JDE" />
```

Attributes

Attribute	Description	Value
<i>sourceClass</i>	Required. The class name of the input object for the data transformation.	The name of a valid object and property.
<i>targetClass</i>	Required. The class name of the output object for the data transformation.	The name of a valid object and property.
<i>sourceDocType</i>	Optional. When the input object is a virtual document, this string identifies its DocType.	A string.
<i>targetDocType</i>	Optional. When the output object is a virtual document, this string identifies its DocType.	A string.
<i>language</i>	Optional. Should be objectscript	objectscript
<i>create</i>	Optional. The create option desired for the target object. If not specified, the default is new.	This can take one of the following values: new, copy, or existing as detailed in the following description.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <transform> element.
Most activities	Optional. <transform> may contain zero or more of the following elements in any combination: <assign>, <code>, <foreach>, <if>, <sql>, <subtransform>, or <trace>.

Description

The <transform> element is the outermost element for a DTL document. All the other DTL elements are contained within a <transform> element. Within the <transform>, the two objects have the names *source* and *target*, respectively. For example:

XML

```
<transform targetClass='Demo.DTL.ExampleTarget'
          sourceClass='Demo.DTL.ExampleSource'
          create='new'
          language='objectscript'>
  <trace value='"Convert from lowercase to uppercase"' />
  <assign property='target.UpperCase'
        value='$ZCONVERT(source.LowerCase,"U")'
        action='set' />
</transform>
```


Source and Target Objects

The *sourceClass* and *targetClass* may identify standard production message classes, each of which contains a set of properties. If so, the *sourceDocType* and *targetDocType* attributes are not needed.

Alternatively, the *sourceClass* and *targetClass* may identify virtual documents. In this case the *sourceDocType* and *targetDocType* attributes are needed to tell InterSystems IRIS which message structure to expect in the virtual document.

Values for the create Option

The *create* option for the target object may have one of the following values:

- **new**—Create a new object of the target type, before executing the elements within the data transformation. This is the default.
- **copy**—Create a copy of the source object to use as the target object, before executing the elements within the transform.
- **existing**—Use an existing object, provided by the caller of the data transformation, as the target object.

DTL <true>

Performs a set of activities when the condition for an <if> element is true, within a [DTL transformation](#).

Syntax

```
<if condition="1">
  <true>
    ...
  </true>
  <false>
    ...
  </false>
</if>
```

Attributes

None.

Elements

Element	Purpose
<annotation>	Optional. A text string that describes the <true> element.
Most activities	Optional. <true> may contain zero or more of the following elements in any combination: <assign>, <code>, <foreach>, <if>, <sql>, <subtransform>, or <trace>.

Description

A <true> element is used within an [<if>](#) to contain elements that need to be executed if the condition is true.

A

Configuring the DTL Explainer

Each DTL has a description field meant to help other users understand what the DTL does. You can configure your system to include a **Generate** button that launches an AI tool (the DTL Explainer), which examines the DTL logic and generates a detailed description. The user can include this generated text in the description field, along with manually entered text if wanted, and the resulting text is fully editable.

A.1 On-prem Systems

To configure your system to include the DTL Explainer:

1. Obtain a key for OpenAI.
2. In an ObjectScript shell, enter the following commands:

```
set sc = ##class(Security.Resources).Create("DTLExplainResource")
set sc = ##class(%Wallet.Collection).Create("%DTLExplain", { "Resource": "DTLExplainResource" })
set sc = ##class(%Wallet.KeyValue).Create("%DTLExplain.Key", { "Secret": {"token": "YourOpenAIKeyHere"
}, "Usage": ["HTTP"] })
```

After these changes, the **Generate** button will be visible in the DTL Editor, above the **Description** field for the DTL.

A.2 Cloud-based Systems

If you want your system to include the DTL Explainer, create an iService ticket requesting this.

A.3 See Also

- [Using the DTL Explainer](#)

